

Writing Story Flows

Introduction

Story flows are structured narratives that describe how users interact with software applications to accomplish specific tasks. They provide a clear, step-by-step breakdown of user interactions, system responses, and alternative paths that can occur during the execution of a feature.

What Are Story Flows?

Story flows document the complete flow a user takes when performing actions in an application. They serve as blueprints for developers, testers, and stakeholders to understand exactly how features should behave under different conditions.

Key characteristics:

1. Written from the user's perspective
2. Include both normal and exceptional scenarios
3. Detail system responses to user actions
4. Account for validation, error handling, and edge cases

Core Components of Story Flows

1. User Story Statement

Every story flow begins with a clear statement of what the user wants to accomplish:

"As an application user, I want to [action] so that [benefit/goal]"

2. Trigger

The specific action or event that initiates the flow:

- HTTP requests (GET, POST, PUT, DELETE)
- User interface interactions
- System events or conditions

3. Flow Steps

A numbered sequence describing the interaction between user and system:

- User actions (inputs, clicks, submissions)
- System responses (displays, redirects, validations)
- Data processing steps

4. Guards and Exception Flows

Guards are conditions that must be met for the flow to continue normally. When guard conditions fail, **exception flows** define alternative paths:

- Data validation checks
- Authentication/authorization requirements
- Resource existence verification

5. Alternative Flows (Alt)

Different paths the user can take based on choices or system states:

- User decisions (yes/no confirmations)
- Different validation outcomes
- Optional steps or features

Writing Effective Story Flows

Structure and Organization

Use consistent formatting:

- Number main flow steps sequentially
- Use sub-numbering for alternative and exception flows (5.1, 5.2, etc.)
- Indent nested flows for visual clarity

Be specific about endpoints and actions:

- Include exact URL patterns (`/book/details/{id}`)
- Specify HTTP methods (GET, POST)
- Name specific views or pages

Language and Clarity

Write in active voice:

- "The application responds with..." rather than "A response is given"
- "I enter the new book data" rather than "Data is entered"

Use precise terminology:

- Distinguish between different types of system responses (redirect vs. display)
- Be clear about data flow (save to database, populate form, show errors)

Maintain user perspective:

- Start with user actions when appropriate
- Clearly separate user steps from system responses

Handling Complexity Guards and validation:

- Place guard conditions early in the flow
- Clearly define what happens when guards fail
- Include "Exit flow" statements when processes terminate early

Error handling:

- Account for both validation errors and system errors
- Specify what information is preserved (populated forms)
- Define user feedback mechanisms (error messages)

Alternative paths:

- Use "Alt:" to introduce decision points
- Number alternatives consistently (3.1, 3.2)
- Ensure all paths eventually converge or exit cleanly

Well-written story flows serve as living documentation that bridges the gap between user requirements and technical implementation. They ensure that all stakeholders have a shared understanding of how features should behave and provide a roadmap for development, testing, and user experience design.

Remember that story flows are iterative documents that should evolve with your application. Regular review and updates ensure they remain accurate and useful throughout the development lifecycle.

Common Patterns in CRUD Operations

Create Operations

1. GET request to display form
2. User data entry
3. POST request with data
4. Validation check (guard)
5. Success: save and redirect
6. Failure: redisplay form with errors

Read Operations

1. GET request with identifier
2. Existence check (guard)
3. Success: display data
4. Failure: redirect or show error

Update Operations

1. GET request to display populated form
2. Existence check (guard)
3. User modifies data

4. POST request with updated data
5. Validation and save process
6. Appropriate response based on outcome

Delete Operations

1. GET request to initiate deletion
2. Existence check (guard)
3. Confirmation request
4. User decision (Alt: yes/no)
5. Execute or cancel based on choice
6. Redirect to appropriate view

Examples

As an application user, I want to view a list of all books.

Trigger:

I send a GET request to `/book/index`

Flow:

1. The application responds with the list of all books.

As an application user, I want to create a new book.

Trigger:

I send a GET request to `/book/create`

Flow:

1. The application responds with the create book view.
2. I enter the new book data.
3. I save the new book data.
4. The application sends a POST request to `/book/create`.
5. **Guard:** [The data is valid]
 - 5.1. **Exception flow:**
 - 5.1.1. The application does not save to the database
 - 5.1.2. The application responds with the populated create book view
 - 5.1.3. The application shows any error messages
 - 5.1.4. Exit flow
6. The application saves the data to the database
7. The application redirects to `/book/index`

As an application user, I want to view a book's information.

Trigger:

I send a GET request to `/book/details/{id}`

Flow:

1. **Guard:** [id exists]
 - 1.1. Exception flow:
 - 1.1.1. The application redirects to `/book/index`
2. The application responds with the book details.

As an application user, I want to update a book's data.

Trigger:

I send a GET request to `/book/update/{id}`

Flow:

1. **Guard:**[id exists]
 - 1.1. **Exception flow:**
 - 1.1.1. The application redirects to `/book/index`
2. The application responds with the update book view
3. I enter the updated data
4. I save the updated data
5. The application sends a POST request to `/book/update`
6. **Guard:** [The data is valid]
 - 6.1. **Exception flow:**
 - 6.1.1. The application does not save to the database
 - 6.1.2. The application responds with the update book view
 - 6.1.3. The application shows any error messages
 - 6.1.4. Exit flow
7. The application saves the data to the database
8. The application redirects to `/book/index`

As an application user, I want to delete a book's data.

Trigger:

I send a GET request to `/book/delete/{id}`

Flow:

1. **Guard:**[id exists]
 - 1.1. **Exception flow:**
 - 1.1.1. The application redirects to `/book/index`
2. The application confirms that I want to delete the book's data
3. **Alt:**
 - 3.1. [I answer yes]
 - 3.1.1. The application deletes the book data from the database
 - 3.2. [I answer no]

3.2.1. The application does not delete the book data from the database
4. The application redirects to `/book/index`

Exercise: Practice Writing Story Flows

Assignment: Choose a simple web application feature (such as user registration, password reset, or product search) and write a complete story flow following the patterns and guidelines discussed in this lecture.

Requirements:

1. Include a clear user story statement
2. Define the trigger action
3. Write the main flow with numbered steps
4. Add at least one guard condition with exception flow
5. Include at least one alternative flow
6. Ensure all paths have clear endpoints